National University of Singapore
School of Computing
CS3244: Machine Learning
Solution to Tutorial 4 (1 week)

Overfitting, Regularization and Validation

Contents:
Tutorial 4: Overfitting, Regularization and Validation

Readings: *Learning From Data*, Chapter 4.

1. **Maximum A Posteriori (MAP) Estimation.**

Suppose a random variable x follows a distribution $P(x|\lambda)$, where λ, the hyper-parameter(s) of the distribution, is also a random variable. Now we have a set of realizations of x, $X = \{x_1, x_2, ..., x_n\}$, independently generated from $P(x|\lambda)$. Besides, we also know from experience that λ follows a distribution $P(\lambda)$. Given X and $P(\lambda)$, we want to know what the most probable distribution $P(x|\lambda)$ is that generates X. In other words, we want to compute the most probable value of λ that fits the data X.

One way to do this is to compute the λ value that maximizes the posterior distribution $P(\lambda|X)$. Based on Bayes' rule,

$$P(\lambda|X) = \frac{P(X|\lambda)p(\lambda)}{P(X)} \tag{1}$$

$$= \frac{P(\lambda)\prod_{i=1}^{N} P(x_i|\lambda)}{P(X)} \qquad x_i \text{ is i.i.d.} \tag{2}$$

Hence, the most probable value of λ is:

$$\lambda_{\text{MAP}} = \arg\max_{\lambda} P(\lambda|X) \tag{3}$$

$$= \arg\max_{\lambda} \frac{P(\lambda)\prod_{i=1}^{N} P(x_i|\lambda)}{P(X)} \tag{4}$$

$$= \arg\max_{\lambda} P(\lambda)\prod_{i=1}^{N} P(x_i|\lambda) \qquad \text{P(X) is a constant; doesn't affect finding argmax.} \tag{5}$$

Using the aforementioned procedure to compute the most probable value of λ is called MAP Estimation. The intuitive idea behind MAP estimation is to combine the prior knowledge ($P(\lambda)$) with the data we actually get (X), to estimate the value of λ.

Now, let's practice MAP estimation. Suppose x follows a geometric distribution:

$$P(x|\lambda) = (1-\lambda)^{x-1}\lambda, \quad \text{for } x = 0, 1, 2, 3, ... \tag{6}$$

and λ is a continuous random variable with the following probability density function:

$$P(\lambda) = \begin{cases} 2\lambda, & 0 \le \lambda \le 1; \\ 0, & \text{otherwise.} \end{cases} \tag{7}$$

We have 1 sample of x drawn from $P(x|\lambda)$:

$$x = 2 \tag{8}$$

What is the MAP estimation of λ?

$$\begin{aligned}
\lambda_{MAP} &= \arg\max_{\lambda} P(\lambda) \prod_{i=1}^{N} P(x_i|\lambda) && \textit{From Eq. (5)} && (9)\\
&= \arg\max_{\lambda} P(\lambda) \prod_{i=1}^{N} P(2|\lambda) && \textit{Substitute in only sample} && (10)\\
&= \arg\max_{\lambda} 2\lambda[(1-\lambda)^2\lambda] && \textit{Eqs. (6) and (7)} && (11)\\
&= \arg\max_{\lambda} (2\lambda)(1-\lambda)(\lambda) && && (12)\\
&= \arg\max_{\lambda} 2\lambda^2 - 2\lambda^3 && && (13)
\end{aligned}$$

Take the derivative of Eq. (13) and set it to 0 to solve for a maximum:

$$\begin{aligned}
\textit{Solve } 4\lambda - 6\lambda^2 &= 0 && (14)\\
0 &= (2\lambda)(2-3\lambda) && (15)\\
\lambda = 0 \textit{ or } \lambda &= \frac{2}{3} && (16)
\end{aligned}$$

Take max λ, i.e., $\lambda = \frac{2}{3}$ as the answer.

2. **Cross Validation.** Consider a learning model that takes $N^2 \log N$ seconds of training when using N examples.

   (a) what is the total amount of time (in seconds) needed when running leave one out cross validation on 30 such models with different parameters to get the final hypothesis?

   (b) what is the total amount of time (in seconds) needed when running 10-fold cross validation on 30 such models with different parameters to get the final hypothesis?

First, let us make sure we are not confused about difference between "models" and "examples". The examples are the data set, the amount of points used to produce a hypothesis. We can use cross validation to produce different hypotheses h per "model". Model, here, could refer to different settings of hyperparameters (such as the regularization constant λ; as in lecture's Slide 31, where the number of models M, equals 30. Usually when doing this type of model selection, we hope to have $|M|$ relatively small, such that the search for the proper model is not particularly sensitive, and won't result in further overfitting of the hyperparameters.

For Part A, when using LOOCV, for each model, we partition the data to N parts, taking $N-1$ for training and 1 for validation orderly. Hence, we need to train with $N-1$ examples for N times for each model. Therefore, the total time we need is:

$$30N(N-1)^2 \log(N-1) \tag{17}$$

For Part B, when using 10-fold cross validation, we partition the data to 10 *parts, taking* 9 *parts for training and* 1 *part for validation. Hence, we need to train with* $\frac{9}{10}N$ *examples for 10 times, validate using the last part, and repeat this process ten times, rotating the part used for validation. Therefore, the time we use to select the best model is:*

$$30 \cdot 10 \cdot (\frac{9}{10}N)^2 \log(\frac{9}{10}N) \tag{18}$$

To get the final hypothesis, we use all the N *examples to train on the model we selected, which takes* $N^2 \log N$ *seconds. Hence, the total time we need is:*

$$30 \cdot 10 \cdot (\frac{9}{10}N)^2 \log(\frac{9}{10}N) + N^2 \log N = 243N^2 \log(\frac{9}{10}N) + N^2 \log N \tag{19}$$

You may be wondering why we have the extra term at the end of Eq. (19) but not at the end of Eq. (18). This is because for large N*, there probably won't be much difference for a model trained with* N *or* $N-1$ *examples. Of course, you may prefer to just pay the extra (N+1)*th *iteration over the slightly larger* N *example dataset to ensure you have an optimum hypothesis* $g(\mathbf{x})$*.*

3. **Cross Validation (Programming).** This exercise gives you additional practice on using SVMs, extended with cross-validation, similar to Homework #2. Wine Quality[1] are two datasets related to red and white variants of the Portuguese "Vinho Verde" wine. In this task, you need to predict the wine's quality (score in {0,1,2,...,10}) based on the attributes of the wine. This is a classification problem with 11 classes. To save the training time, we have subsampled 2000 examples from the original white wine dataset as the training data (see `train.txt`) and 500 examples as the test data (see `test.txt`) for this programming exercise. The last column in the file is the correct label. You are required to write a program using SVM to solve this classification problem. Use 0/1 error to compute the accuracy. Try all the combinations of the following hyperparameters to get different models:

   - kernel $\in$ {linear, polynomial, RBF}
   - penalty parameter $C \in \{0.1, 1, 10\}$

   For the polynomial kernel, use a degree of 2, and the constant coefficient `coef0` $= 0$. For RBF kernel, use $\gamma = 0.001$.

   For each model, run a 5-fold cross validation. Select the model with the highest average cross validation accuracy. Feed all training data to this model to get a final hypothesis. You don't need to shuffle the data when doing cross validation since it is already randomized.

   (a) Output the accuracy for each fold in cross validation for each model. Output the average accuracy in cross validation for each model.

   (b) Output hyper-parameters of the model that performs best in cross validation and its corresponding average cross validation accuracy

   (c) Output the accuracy for test data using the model trained with all training data.

   (Hint: You can use sklearn to do SVM classification and cross validation, see SVM classification[2] and cross validation[3])

[1] `https://archive.ics.uci.edu/ml/datasets/Wine+Quality`
[2] `http://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html#sklearn.svm.SVC`
[3] `http://scikit-learn.org/stable/modules/cross_validation.html`

Note that the Scikit Learn package gives you a single, one-line command to run cross validation, so it is pretty simple to embed practical model selection in your work. However, you should be familiar with the steps to do cross validation, as it is important to be able implement to modify cross validation on your own, when you need to modify the procedure. We sketch the pseudocode for cross validation.

(a) *Partition the datasets into its K folds.*

(b) *Initialize E_cv to 1, and $best_fold$ to nil.*

(c) *Run a loop for the training and testing of the K models that you are going to derive:*

  i. *Training: Train using $K-1$ folds of the data, reserving the Kth fold for validation.*

  ii. *Validate: Test using the Kth fold. Evaluate and determine E_val.*

  iii. *If E_val is better than the currect E_cv, update E_cv and $best_fold$.*

(d) *Train for $g(x)$: Train using all N examples to obtain $g(x)$.*

(e) *Return E_cv and $g(x)$.*

See `cross_validation.py` *for the code.*

See `result.txt` *for the results given by the code (note that your results may not be the same with the ones given).*

4. L_2 **Regularization and MAP Estimation (harder).**

Attempt this question only after doing Question 1.

In Question 6, Tutorial 2, we have seen how to view linear regression with i.i.d. Gaussian noise from a probabilistic perspective (Maximum likelihood estimation). We proved that, for a linear regression model, $y = \mathbf{w}^\mathbf{T}\mathbf{x} + \epsilon_t$, when the noise $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$ and is i.i.d. for each t, maximizing the log likelihood is equivalent, so far as determining w is concerned, to minimizing LMS in linear regression.

Now, let's see how we can view L_2 regularized linear regression from a probabilistic perspective. As in Tutorial 2, we keep the assumption that $y = \mathbf{w}^\mathbf{T}\mathbf{x} + \epsilon_t$, $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$ and is i.i.d. for each t. We introduce a prior distribution over the parameter $\mathbf{w}$. Concretely, we assume $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \tau^2\mathbf{I})$, where $\mathbf{I}$ is the identity matrix.

Prove that computing the optimal $\mathbf{w}$ for this L_2 regularized linear regression is equivalent to maximizing the posterior distribution of $\mathbf{w}$. What is the regularization parameter λ in this case? Assume the data are drawn independently from $P(y|\mathbf{x}, \mathbf{w})$. (Hint: $P(\mathbf{w}|\mathbf{X}) = P(\mathbf{w})$)

For background, this question relates what we learned about in regularization to another form of machine learning that we don't cover in this course, Bayesian inference – responsible for the machine learning formalism known as probabilistic graphical models (PGM), which is another formalism for machine learning. As we hinted at the beginning of the course, Machine Learning is a multidisciplinary field, and the Bayesian viewpoint is an important one, so important that it is covered in another NUS module: CS 5340 Uncertainty Modeling in AI. Thus, it is not covered in this courses' lecture.

This question is meant for you to be familiar with the idea behind the Bayesian viewpoint, so that if you go farther in your machine learning journey, you'll be ready to understand what you'll see then[4].

In any case, the Bayesian model has some very nice properties that we should be familiar with. Wikipedia states this very well[5]*, where we have added a few things in* **bold***:*

> *The critical point about Bayesian inference, then, is that it provides a principled way of combining new evidence with prior beliefs* **(conditional probabilities with priors, such as** $P(y|X)P(X)$**)***, through the application of Bayes' rule. (Contrast this with frequentist inference* **(just listening to the data, like Maximum Likelihood Estimation (MLE); conditional probabilities such as** $P(y|X)$**)***, which relies only on the evidence as a whole, with no reference to prior beliefs.)*
>
> *Furthermore, Bayes' rule can be applied iteratively: after observing some evidence, the resulting posterior probability can then be treated as a prior probability, and a new posterior probability computed from new evidence. This allows for Bayesian principles to be applied to various kinds of evidence, whether viewed all at once or over time. This procedure is termed "Bayesian updating".*

In this problem, we see that the prior assumption that we make in adopting L_2 regularization (in lecture, (squared) "weight decay") is the same as the Bayesian method of MAP estimation where the weight vector follows a multivariate normal distribution.

Anyways, enough of the preamble. Let's solve the problem.

—

Let's first define some notations. Let $\mathbf{X} = (\mathbf{x_1}, \mathbf{x_2}, ...\mathbf{x_N})^T$ *represent N input values and* $\mathbf{y} = (y_1, y_2, ..., y_N)^T$ *represent their corresponding target values.*

We apply Bayes' rule to get the posterior probability distribution:

$$P(\mathbf{w}|\mathbf{X}, \mathbf{y}) = \frac{P(\mathbf{y}|\mathbf{X}, \mathbf{w}) \cdot P(\mathbf{w}|\mathbf{X})}{P(\mathbf{y}|\mathbf{X})} \tag{20}$$

$$= \frac{P(\mathbf{y}|\mathbf{X}, \mathbf{w}) \cdot P(\mathbf{w})}{P(\mathbf{y}|\mathbf{X})} \qquad \textit{Apply the hint: } P(\mathbf{w}|\mathbf{X}) = P(\mathbf{w}) \tag{21}$$

$$= \eta \cdot P(\mathbf{y}|\mathbf{X}, \mathbf{w}) \cdot P(\mathbf{w}) \tag{22}$$

where η is a normalization constant, which normalizes by summing over all of the terms $P(\mathbf{y}|\mathbf{X}, \mathbf{w}) \cdot P(\mathbf{w})$*. Note that the hint* $P(\mathbf{w}|\mathbf{X}) = P(\mathbf{w})$ *is a usualy assumption in machine learning; i.e., we assume that the model* $\mathbf{w}$ *is not influenced by the data* $\mathbf{X}$*.*

This equation has two non-trivial terms, $P(\mathbf{y}|\mathbf{X}, \mathbf{w})$ *and* $P(\mathbf{w})$*. We'll look at the terms in sequence, and after deriving the form for each, we'll put them together.*

For the first term, $P(\mathbf{y}|\mathbf{X}, \mathbf{w})$*, since* $y = \mathbf{w^T x} + \epsilon_t$*,* $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$ *and* ϵ_t *is i.i.d., we have:*

$$y|\mathbf{x}, \mathbf{w} \sim \mathcal{N}(\mathbf{w^T x}, \sigma^2), \tag{23}$$

[4]Note that other universities and institutions may cover PGMs and the Bayesian viewpoint; introductory machine learning does not fit very well into our NUS semester schedule, we have purposely cut this view from our course to deal with our time constraints.

[5]`https://en.wikipedia.org/wiki/Bayesian_inference`

and because the data are drawn from $P(y|\mathbf{x}, \mathbf{w})$ independently, we have:

$$P(\mathbf{y}|\mathbf{X}, \mathbf{w}) = \prod_{i=1}^{N} P(y_i|\mathbf{x_i}, \mathbf{w}) \tag{24}$$

$$= \prod_{i=1}^{N} \frac{1}{\sqrt{2\pi}\sigma} \exp\left\{\frac{-1}{2\sigma^2}(y_i - \mathbf{w^T x_i})^2\right\} \tag{25}$$

$$= \left(\frac{1}{\sqrt{2\pi}\sigma}\right)^N \exp\left\{\frac{-1}{2\sigma^2}\sum_{i=1}^{N}(y_i - \mathbf{w^T x_i})^2\right\} \tag{26}$$

where Eq. (26) is the result of applying the definition of the probability density function for the normal distribution[6] *for the univariate case (it is univariate (= of a single variable) because the value $\mathbf{w^T x}$ is a scalar value; sum of inner products).*

For the second term, the assumed prior, we are told:

$$\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \tau^2\mathbf{I}). \tag{27}$$

This is a multivariate (= more than one variable) normal distribution[7]*, which has density:*

$$\frac{1}{\sqrt{(2\pi)^k|\Sigma|}} exp(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T\Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})$$

and is denoted as $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \Sigma)$ where Σ is the covariance matrix, and $|\Sigma|$ is the determinant of the covariance matrix. In our case $\Sigma = \tau^2\mathbf{I}$, which is means that we have a $k \times k$ matrix where the covariate terms are 0 (i.e., there is no correlation between the weights w_i). To be clear, k is the dimensionality of $\mathbf{w}$.

Substituting $\mathbf{w}$ for $\mathbf{x}$, $\mathbf{0}$ for μ and $\tau^2\mathbf{I}$ for Σ in the definition of the multivariate normal, we have:

$$P(\mathbf{w}) = (2\pi)^{-\frac{k}{2}}|\tau^2\mathbf{I}|^{-\frac{1}{2}} \exp\left\{-\frac{1}{2}(\mathbf{w} - \mathbf{0})^T(\tau^2\mathbf{I})^{-1}(\mathbf{w} - \mathbf{0})\right\} \tag{28}$$

$$= (2\pi)^{-\frac{k}{2}}(\tau^{2k})^{-\frac{1}{2}}|\mathbf{I}|^{-\frac{1}{2}} \exp\left\{-\frac{1}{2}\mathbf{w}^T(\frac{1}{\tau^2})(\mathbf{I})^{-1}\mathbf{w}\right\} \qquad \textit{Factoring out } \tau^2 \tag{29}$$

$$= (2\pi\tau^2)^{-\frac{k}{2}} \exp\left\{-\frac{1}{2\tau^2}\mathbf{w}^T\mathbf{w}\right\} \qquad \textit{Simplifying out } \mathbf{I} \tag{30}$$

Now we are ready to put it together.

$$P(\mathbf{w}|\mathbf{X}, \mathbf{y}) = \eta P(\mathbf{y}|\mathbf{X}, \mathbf{w}) \cdot P(\mathbf{w}) \tag{31}$$

$$= \eta \left(\frac{1}{\sqrt{2\pi}\sigma}\right)^N \exp\left\{\frac{-1}{2\sigma^2}\sum_{i=1}^{N}(y_i - \mathbf{w^T x_i})^2\right\}(2\pi\tau^2)^{-\frac{k}{2}} \exp\left\{-\frac{1}{2\tau^2}\mathbf{w}^T\mathbf{w}\right\} \tag{32}$$

[6] `https://en.wikipedia.org/wiki/Normal_distribution`
[7] `https://en.wikipedia.org/wiki/Multivariate_normal_distribution`

Maximizing the posterior with respect to $\mathbf{w}$*, is equivalent to maximizing the log posterior. Hence,*

$$\begin{aligned}
\mathbf{w}^* &= \arg\max_{\mathbf{w}} P(\mathbf{w}|\mathbf{X},\mathbf{y}) && (33)\\
&= \arg\max_{\mathbf{w}} \log P(\mathbf{w}|\mathbf{X},\mathbf{y}) && (34)\\
&= \arg\max_{\mathbf{w}} \left\{ \log\left[\eta\left(\frac{1}{\sqrt{2\pi}\sigma}\right)^N (2\pi\tau^2)^{-\frac{k}{2}}\right] - \frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i - \mathbf{w^T x_i})^2 - \frac{1}{2\tau^2}\mathbf{w}^T\mathbf{w}\right\} && (35)\\
&= \arg\min_{\mathbf{w}} \left\{ \frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i - \mathbf{w^T x_i})^2 + \frac{1}{2\tau^2}\mathbf{w}^T\mathbf{w}\right\} && (36)\\
&= \arg\min_{\mathbf{w}} \left\{ \frac{1}{2}\sum_{i=1}^{N}(y_i - \mathbf{w^T x_i})^2 + \frac{\sigma^2}{2\tau^2}\mathbf{w}^T\mathbf{w}\right\} && (37)
\end{aligned}$$

where Eq. (36) results by dropping the first term in Eq. (35) as it is a constant with respect to $\mathbf{w}$*, and factors the minus signs in the second and third terms out, by changing the argmax to an argmin. The form of Eq. (37) should be familiar to you; the first term is the standard LMS regression term, and the second term is equivalent to the weight decay (*L_2*) regularization term we introduced in the regularization lecture.*

This is equivalent to compute $\mathbf{w}$ *in* L_2 *norm regularized linear regression, with a regularization parameter given by* $\lambda = \frac{\sigma^2}{\tau^2}$*. This shows the connection between maximum a posteriori estimation in Bayesian inference and regularization;* L_2 *norm regularization of* $\mathbf{w}$ *is equivalent to placing a simple prior probability distribution of a multivariate normal distribution on* $\mathbf{w}$*.*

L_2 *norm regularization was also derived by statisticians independently of the machine learning community, so if you hear the term "ridge regression" it is sometimes meant to denote* L_2 *norm regression (although it more properly refers to regression using Tikhonov regularization, of which the* L_2 *form is the most widely used).*